CLIO Search: Agentic Scientific Data Discovery with Intelligent Orchestration

Author A and Author B
Department of Computer Science
University Name, City, Country
{authora, authorb}@university.edu

***Abstract*—Scientific data discovery requires search that thinks before it acts. Before executing a query, an effective search system must profile each corpus to understand what data and metadata exist. It must decide which search strategies apply, activating science-aware operators when structured measurements are present and falling back to text retrieval when they are not. It must execute domain-specific reasoning, such as the arithmetic needed to recognize that 200 kPa, 200000 Pa, and 0.2 MPa describe identical pressure. And when results are insufficient, it must refine its approach. This observe-decide-act-evaluate cycle is what distinguishes agentic search from conventional retrieval. Few current systems perform these operations in combination.**

We present CLIO Search, an agentic retrieval system for scientific data. CLIO implements a four-stage pipeline: Discover (profile each corpus in 14–22 ms), Reason (select retrieval branches based on corpus characteristics), Transform (execute dimensional conversion and formula normalization as deterministic database operators alongside BM25 and dense vector search), and Execute (federate across storage backends with iterative refinement via an LLM rewriter or deterministic fallback achieving 96% of LLM-assisted quality).

CLIO achieves 0.99 P@5 on cross-unit queries (BM25: 0.40, NC-Retriever: 0.16); same-unit controls confirm comparable performance across all methods (0.30–0.45). Precision improves by 87% on real NOAA data. At scale on 2.5M arXiv abstracts across 4 GH200 nodes: sub-50 ms latency, 0.94 weak scaling efficiency. All code and benchmarks are open-source.

I. Introduction

When a scientist searches for data across a large repository, they do not simply type a query and read results. They inspect what each dataset contains, determine whether it has structured metadata or only raw files, decide which search approach will work for each dataset, apply domain knowledge to interpret measurements across different conventions, and refine their search when initial results are insufficient. This process is reasoning, not retrieval. It is how a human expert navigates heterogeneous scientific data. No retrieval system replicates it.

At HPC scale, human reasoning does not scale. A single exascale job on Frontier can produce 80 PB of simulation output. A commonly cited estimate holds that 60 to 80% of scientific data is never reused, not for lack of access, but for lack of search [1]. The FAIR data principles, now mandated by NSF, NIH, and DOE, require that data be Findable, Accessible, Interoperable, and Reusable, yet automated assessments show no major repository achieves 50% reusability compliance. AI agents are now entering this space: Coscientist [2] executes lab synthesis autonomously, AI Scientist v2 [3] generates research end-to-end, and autonomous agents at ORNL [4] orchestrate experiments across national facilities. But ScienceAgentBench [5] reports 32.4% end-to-end task success, with failures traced to data handling rather than reasoning. A comprehensive survey [6] identifies scientific data management as explicitly underexplored. These agents inherit the same retrieval infrastructure that fails human scientists: infrastructure that treats all content as plain text.

Current retrieval systems fall short of scientific data discovery in three fundamental ways.

First, scientific search requires domain reasoning that retrieval systems do not perform. A query for pressures between 190 and 360 kPa must recognize that "200000 Pa" and "0.2 MPa" are both matches, but this requires arithmetic, not text similarity. Formulas compound the problem: "$F = ma$," "$F = m \cdot a$," and "$ma = F$" express the same physical law but produce distinct token sequences that defeat both keyword and embedding retrieval. The Numeracy Gap study [7] evaluates 13 embedding models and reports 0.54 binary retrieval accuracy on numerical content. String normalization [8] cannot bridge SI prefixes. Learned approaches [9], [10] achieve up to 65% recall improvement but provide no correctness guarantee. The consequences are real: NASA's $327M Mars Climate Orbiter was lost because one system used pound-seconds while another used newton-seconds; the Gimli Glider incident stemmed from kg vs. lb fuel loading; and an estimated 7,000 to 9,000 medication deaths per year in US healthcare result from unit errors.

Second, scientific datasets vary enormously in metadata quality, and no search system adapts to this variation. Automated FAIR metadata assessments consistently find that fewer than half of datasets in major repositories carry structured, machine-readable annotations [1]. An HDF5 file with Climate and Forecast convention attributes provides explicit units for every variable; a raw binary dump on a parallel filesystem may offer nothing beyond a filename. A fixed retrieval pipeline cannot handle both: it either wastes computation on datasets with no extractable measurements or misses opportunities when rich metadata is available. A search system that profiles each corpus before querying, determining what metadata exists and which strategies are viable, would address this gap, but no current retrieval system provides this capability.

Third, scientific data is distributed across storage systems

that no search engine spans with domain awareness. A single experimental campaign may deposit raw output on a parallel filesystem, preprocessed arrays in HDF5, derived features in a vector database, and catalog entries on an external platform. Existing systems either federate generic text search without science-aware operators [11], or capture storage provenance without exposing a search interface [12]. No existing system combines all three: domain-aware operators, corpus-adaptive orchestration, and federated HPC access.

These challenges share a common requirement: the search system must reason about *how* to search before it executes the search. It must observe what each corpus contains, decide which strategies to apply, act with domain-specific operators, and evaluate whether to refine. This observe-decide-act-evaluate cycle is what separates agentic search from conventional retrieval. It is how human scientists search. No retrieval system implements it.

We present CLIO Search, a retrieval system that implements this cycle for scientific data. CLIO profiles each corpus to discover what metadata exists (completing in 14–22 ms, scaling sub-linearly to thousands of namespaces via SQL aggregation on indexed columns), selects retrieval branches based on corpus characteristics (skipping branches that cannot contribute), executes science-aware operators as deterministic database predicates complementing BM25 and dense vector search, and federates across heterogeneous storage backends. An iterative refinement loop expands, narrows, or pivots the query based on intermediate results; the loop is driven by an LLM query rewriter in connected environments or a deterministic fallback for air-gapped HPC deployments, where the fallback achieves 96% of LLM-assisted P@5 quality. Domain knowledge enters the pipeline as executable, verifiable computation: the fact that 5 km = 5000 m is an algebraic invariant encoded in the retrieval operator, guaranteed correct for all supported unit transformations. Cross-unit queries constitute a significant fraction of scientific data discovery: pressure alone is expressed in seven incompatible units across disciplines (Pa, hPa, kPa, MPa, bar, atm, psi), temperature in three scales (K, °C, °F), and energy in seven variants (eV, keV, MeV, GeV, kJ, MJ, GJ).

We make three contributions:

1) **Science-aware retrieval operators.** We introduce dimensional conversion and formula normalization (algebraic equivalence matching, e.g., $F{=}ma$ vs. $ma{=}F$) as deterministic, first-class retrieval branches that execute alongside BM25 and dense vector search. A composable unit registry based on dimensional analysis canonicalizes measurements to SI base units at index time, enabling cross-unit queries as database range predicates with correctness guaranteed for all supported unit transformations. The registry covers 46 units across 12 physical domains[1]; adding a new unit requires one configuration entry, not code changes.
2) **Corpus-adaptive search orchestration.** A profiling layer inspects each namespace before querying, learning what metadata exists and which retrieval branches are viable. The system adapts its search strategy per dataset: rich-metadata corpora activate science-aware operators, sparse corpora fall back to text-based retrieval, and multi-hop refinement expands, narrows, or pivots the query based on intermediate results.
3) **Federated scientific search.** A connector architecture provides unified search across heterogeneous storage backends (filesystems, object stores, HDF5, NetCDF, IOWarp blob storage, external catalogs) with per-connector capability negotiation. Each connector declares which retrieval branches it supports, and the coordinator routes queries accordingly.

We evaluate on a controlled benchmark (210 documents, 80 queries), real observatory data (288 NOAA GHCN-Daily documents), a live external catalog (341 NDP datasets), distributed HPC infrastructure (2.5M arXiv abstracts across 4 GH200 nodes on NCSA DeltaAI), and IOWarp blob storage (10K blobs). CLIO achieves 0.99 P@5 on cross-unit queries (vs. 0.40 for the strongest text baseline); on same-unit controls all methods score comparably (0.30–0.45), confirming the gains are specific to cross-unit matching. Real-world precision improves by 87% on NOAA data, with sub-50 ms query latency and 0.94 weak scaling efficiency at distributed scale.

II. Background and Related Work

The gap between what retrieval systems deliver and what scientific data discovery demands spans four areas: the hybrid retrieval foundations we build upon, the numeracy crisis that motivates explicit arithmetic operators, quantity-aware retrieval systems that attempt but fail to solve cross-unit matching, and scientific data discovery in HPC.

A. Hybrid Retrieval Foundations

Modern retrieval combines lexical and dense signals. BM25 [13] remains the strongest lexical baseline; dense retrieval via learned bi-encoders [14] captures semantic similarity that keyword matching misses. Bruch et al. [15] show that a learned convex combination consistently outperforms reciprocal rank fusion (RRF) on TREC benchmarks, establishing the analytic framework we adopt. At the agentic frontier, A-RAG [16] introduces hierarchical retrieval interfaces that let LLM agents compose retrieval strategies dynamically.

We build *on* hybrid retrieval, not beside it. Our science-aware operators execute as parallel branches alongside BM25 and dense search within the same fusion pipeline, adding domain-specific recall without replacing any existing component.

B. The Numerical Reasoning Crisis

Numbers are fundamentally broken in both retrieval systems and large language models. The Numeracy Gap study [7] evaluates 13 embedding models on numerical content and reports 0.54 binary retrieval accuracy, barely above random

[1]Length, mass, time, temperature, pressure, velocity, frequency, radioactivity, energy, power, area, and acceleration.

chance. NC-Retriever [10] finds that dense retrieval achieves only 16.3% accuracy on queries with numeric constraints, even when the constraint is stated explicitly in both query and document. The root cause is architectural: tokenizers encode numbers per-digit rather than by magnitude, so "200" and "200000" share no meaningful representation [17]. GSM-Symbolic [18] delivers the sharpest result: changing only the numeric values in grade-school math problems while preserving problem structure causes a 65% accuracy drop, proving that LLMs pattern-match on surface numerals rather than reason about magnitude.

This is not merely a unit problem. Numbers themselves, the substrate of all scientific measurement, are broken in every retrieval and language model. Scientific data retrieval, where matching requires comparing magnitudes across units, is especially vulnerable.

C. Quantity-Aware Retrieval

Three lineages attempt to make retrieval quantity-aware. None succeeds at arithmetic conversion.

String normalization. QFinder [19] extends Elasticsearch with quantity-aware ranking by extracting and normalizing numeric expressions. CQE [20] advances this with dependency parsing over a 531-unit dictionary, handling complex quantity expressions. Numbers Matter! [8] builds a full retrieval pipeline with quantity-aware indexing and evaluation on FinQuant and MedQuant benchmarks. The fundamental limitation persists: string normalization cannot cross SI prefixes. "kilopascal" and "pascal" remain distinct strings after normalization, so a query for "200 kPa" never matches a document containing "200000 Pa."

Learned embeddings. CONE [9] trains a transformer to embed number-unit pairs into a shared vector space, achieving a 25% Recall@10 gain on quantity-bearing queries. NC-Retriever [10] applies two-phase contrastive learning to numeric constraint matching, improving recall@10 by 65.84% over baselines on the NumConQ benchmark. These approaches are probabilistic: a model may learn that 200 kPa is *similar* to 200000 Pa, but provides no correctness guarantee. Weller et al. [21] prove theoretical upper bounds on embedding-based retrieval, showing that fixed-dimensional embeddings cannot faithfully preserve all distance relationships, a result that undercuts learned approaches to exact quantity matching.

Math-aware search. Approach0 [22] matches equation structure via operator trees. SSEmb [23] learns semantic similarity over mathematical expressions. These systems match formula *structure* but are disconnected from physical measurements: they cannot link $F = ma$ to a document reporting force in newtons.

Offline conversion tools. Unit conversion itself is well-established: UDUNITS-2 [24] and GNU Units convert data values during preprocessing, but they do not operate at retrieval time, do not index measurements, and expose no search interface. A scientist must already know which files to convert.

Gap. No existing retrieval system integrates arithmetic SI conversion as a first-class retrieval operator. Offline tools require locating data first; string normalization and learned embeddings operate within retrieval but cannot perform arithmetic. No system combines quantity-aware retrieval with formula normalization or federated search across HPC backends.

D. Scientific Data Discovery

Paper search. OpenScholar [25] indexes 45 million open-access papers and achieves GPT-4o-level citation quality with a specialized 8B-parameter model. HiPerRAG [26] scales RAG to 3.6 million articles across DOE supercomputers Polaris, Sunspot, and Frontier, achieving 90% accuracy on SciQ. Both search *papers*, not experimental data, simulation outputs, or HDF5 files.

Scientific data search. PANGAEA-GPT [27] deploys hierarchical multi-agent search over 400,000+ geoscientific datasets with data-type-aware routing and unit scale validation, achieving 8.14/10 relevance versus a 2.87/10 baseline. LLM-Find [28] uses LLM-generated code to retrieve geospatial data from heterogeneous sources with 80 to 90% success. The distinction is temporal: PANGAEA-GPT validates unit scales *post-hoc* (after retrieval); CLIO converts *at retrieval time* (during indexing and query processing). LLM-Find generates access code per source; CLIO provides reusable retrieval operators. Microsoft Discovery (2025) offers enterprise agentic search with a graph-based knowledge engine over proprietary and external data, but provides no arithmetic unit conversion and is closed-source.

HPC data management. Datum [29] catalogs scientific data at INL but offers no content search. BRINDEXER [30] indexes POSIX metadata on Lustre with 69% query improvement, but provides metadata-only search with no content analysis. PROV-IO+ [12] captures HDF5 I/O provenance with under 3.5% overhead but exposes no search interface. HDF Clinic [31] proposes a knowledge graph over HDF5 but remains unimplemented.

Agentic retrieval. Context-1 [32] trains a 20B-token agentic search agent. PRISM [33] decomposes retrieval into precision/recall subagents. Agentic-R [34] co-trains retrievers with answer-correctness rewards (10 to 13% multi-hop gains). CLADD [35] and SciAgent [36] add domain knowledge operators for drug discovery and scientific workflows. These systems advance orchestration and learned retrieval but none incorporates arithmetic dimensional conversion or formula normalization into the retrieval pipeline. Our work is complementary: PRISM and Agentic-R could be layered on top of our science-aware operators.

Table I summarizes the landscape. Individual capabilities exist in isolation: UDUNITS converts units offline, PANGAEA-GPT validates scales post-hoc, Numbers Matter! indexes quantities as strings, OpenScholar searches papers at scale, NC-Retriever learns numeric embeddings. No prior system integrates arithmetic SI conversion as a first-class retrieval operator, combines it with formula normalization, profiles corpora before querying, or federates across HPC

TABLE I
CAPABILITY COMPARISON ACROSS RETRIEVAL AND SCIENTIFIC DATA SYSTEMS. ✓ = SUPPORTED, ✗ = NOT SUPPORTED, ○ = PARTIAL.

	Our Contributions					General Capabilities				
System	SI Conv.	Formula	Sci. Branches	Federated	HDF5/NetCDF	Agentic	Qty. Extract.	Paper Search	Data Search	LLM Integr.
CLIO (ours)	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
PANGAEA-GPT	✗	✗	✗	✗	✗	✓	○	✗	✓	✓
LLM-Find	✗	✗	✗	○	✗	✓	✗	✗	✓	✓
OpenScholar	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
HiPerRAG	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
Numbers Matter!	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
NC-Retriever	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
CONE	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
Context-1	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
CLADD	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
SciAgent	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓

backends with per-connector capability negotiation. We mark capabilities we lack: CLIO does not index papers. PANGAEA-GPT and LLM-Find receive partial credit for data search and federation respectively, but neither performs arithmetic unit conversion at retrieval time.

III. SYSTEM DESIGN

To address the three failures identified in Section 1, we design CLIO Search around two central principles: science-aware operators must execute as first-class retrieval branches, and an agentic orchestration layer must reason about *what* to search before executing. The architecture comprises a staged pipeline with four parallel branches (Sec. 3.1), dimensional conversion operators that perform explicit SI arithmetic (Sec. 3.2), deterministic formula normalization (Sec. 3.3), a hybrid fusion strategy (Sec. 3.4), federated multi-namespace search (Sec. 3.5), and an agentic orchestration layer with corpus profiling, adaptive branch selection, and multi-hop refinement (Sec. 3.6).

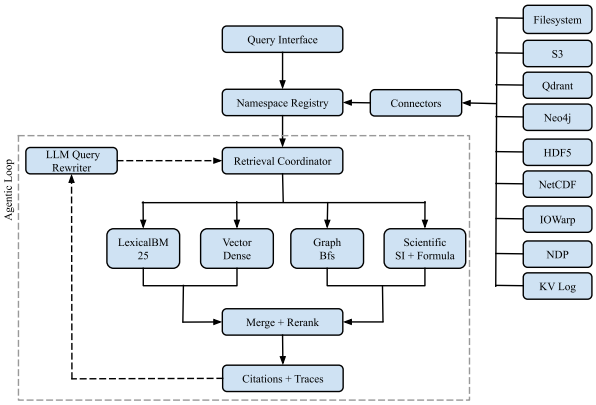

Fig. 1. System architecture of CLIO-AGENTIC-SEARCH. Nine connectors mediate heterogeneous backends; science-aware components execute alongside standard retrieval branches. The agentic loop iteratively refines queries via LLM rewriting.

A. Architecture Overview

The system is a staged pipeline with four parallel retrieval branches (Fig. 1). A query enters through the Query Interface (FastAPI REST endpoint or CLI), which parses the raw query string, extracts typed scientific constraints (numeric ranges, unit filters, formula patterns), and forwards a structured query object to the Namespace Registry. The registry resolves which namespaces—each backed by one of nine connector types—are eligible for the query based on capability negotiation (Sec. 3.5).

The Retrieval Coordinator fans the query out to four parallel branches: lexical (BM25), vector (dense embedding similarity), graph (entity-relationship traversal), and scientific (dimensional measurement and formula queries). Each branch executes independently and returns scored candidate chunks.

The Merge and Rerank stage deduplicates candidates by chunk identifier, applies hard scientific filters, computes a composite score via heuristic fusion, and emits the final top-K results with source citations.

Nine connectors mediate access to heterogeneous backends: local filesystem, S3-compatible object stores, Qdrant vector database, Neo4j graph database, HDF5 (via h5py), NetCDF (via xarray), IOWarp CTE blob storage, NDP catalog (via MCP), and a key-value log store. Each connector declares a capability set specifying which retrieval branches it supports, enabling the coordinator to skip unsupported branches rather than fail. The IOWarp connector indexes scientific metadata from CTE blobs at ingest time and answers cross-unit queries against the DuckDB index; the NDP connector discovers and indexes datasets from the National Data Platform via the Model Context Protocol. Every pipeline stage emits structured trace events for observability.

B. Dimensional Conversion Operators

The core insight behind dimensional conversion is that cross-unit matching is an arithmetic problem, not a similarity problem. When a scientist queries for pressures between 190 and 360 kPa, the system must recognize that a document reporting "250000 Pa" falls within that range. This requires multiplying 190 by 1000 to obtain 190000 Pa and 360 by 1000 to obtain 360000 Pa—a computation that no string normalization or learned embedding can guarantee.

The key insight underlying our approach is that physical units are not a flat enumeration—they are expressions over a small set of base dimensions. Every unit is fully characterized by a *dimension vector* over the seven SI base dimensions $[\mathrm{M}, \mathrm{L}, \mathrm{T}, \mathrm{I}, \Theta, \mathrm{N}, \mathrm{J}]$ (mass, length, time, electric current, temperature, amount of substance, luminous intensity) plus a scale factor and optional offset relative to the SI base unit for that dimension combination. For example, Pa and kPa share the dimension vector $\mathrm{M}^1\mathrm{L}^{-1}\mathrm{T}^{-2}$ with scale factors 1 and 10^3 respectively. Two units are *compatible* if and only if their dimension vectors are equal; *conversion* between compatible units is the ratio of their scale factors.

We implement this as a composable unit registry covering 46 units across 12 physical domains. Table II shows a representative excerpt. Each entry is a 4-tuple: (dimension vector, scale, offset, domain tag). Adding a new unit requires adding a single configuration row—no code changes. The domain tag disambiguates dimensionally identical but physically distinct quantities: becquerel (Bq) and hertz (Hz) are both T^{-1}, but a cross-unit radioactivity query should not match frequency measurements. At index time, a regex-based measurement extractor identifies numeric-unit pairs in document text and scientific file metadata. Each extracted measurement is stored as a 4-tuple: (raw_value, raw_unit, canonical_value, dimension_key). These tuples populate a `scientific_measurements` table in DuckDB, indexed on canonical_value and dimension_key for efficient range scans.

TABLE II
UNIT REGISTRY EXCERPT (8 OF 46 UNITS). EACH UNIT IS A DIMENSION VECTOR + SCALE FACTOR + DOMAIN TAG. ADDING A UNIT = ONE ROW, NO CODE CHANGES.

Unit	Dim. vector	Scale	Domain
Pa	$\mathrm{ML}^{-1}\mathrm{T}^{-2}$	1	pressure
kPa	$\mathrm{ML}^{-1}\mathrm{T}^{-2}$	10^3	pressure
Hz	T^{-1}	1	frequency
Bq	T^{-1}	1	radioact.
eV	$\mathrm{ML}^2\mathrm{T}^{-2}$	1.6×10^{-19}	energy
J	$\mathrm{ML}^2\mathrm{T}^{-2}$	1	energy
m	L	1	distance
km	L	10^3	distance

At query time, three operators execute against the canonicalized measurement store. The **numeric-range** operator converts query bounds to the canonical scale for the query unit's dimension vector and issues a DuckDB range predicate: a query "190:360:kPa" becomes a scan for canonical_value between 190000 and 360000 with dimension_key = $\mathrm{M}^1\mathrm{L}^{-1}\mathrm{T}^{-2}$. Because all pressure units—Pa, hPa, kPa, MPa, bar, atm, psi—share this dimension vector, a single index scan matches measurements originally expressed in *any* pressure unit. The **unit-match** operator filters chunks containing any measurement with a compatible dimension vector. The **value-match** operator retrieves measurements matching a specific canonical value within a configurable tolerance (default $\pm1\%$).

The composability of this design merits emphasis. Becquerel (Bq) and curie (Ci) are both T^{-1} with scale factors 1 and 3.7×10^{10}; adding curie is a single registry entry, and cross-unit radioactivity queries work automatically. Hertz is also T^{-1}, but the domain tag prevents radioactivity queries from matching frequency. The same principle extends to any unit expressible as a product of SI base dimensions—electromagnetic quantities, chemical concentrations, or domain-specific units—without code changes.

Correctness is guaranteed by construction: if the scale factor is correct, the match is correct. String normalization cannot bridge "kilopascal" and "pascal"; learned embeddings [9] achieve only 0.54 binary retrieval accuracy on numerical content [7]. Our guarantee depends on (1) the extractor correctly identifying numeric-unit pairs and (2) the registry covering the unit in question.

C. Formula Normalization

Scientific formulas appear in inconsistent surface forms across documents. "$F = m*a$", "$F = ma$", "$F = m\cdot a$", and "$ma = F$" express the same physical law but produce distinct token sequences that defeat both keyword and embedding retrieval. We normalize formulas through a deterministic six-step pipeline.

First, all whitespace is stripped. Second, all alphabetic characters are lowercased. Third, superscript notations are unified: LaTeX-style `^{N}` and Python-style `**N` are both normalized to `^N`. Fourth, the expression is split on the equality sign

into left-hand and right-hand sides. Fifth, within each side, multiplicative factors are sorted lexicographically, so that `m*a` and `a*m` produce the same canonical form. Sixth, the two sides are rejoined with a normalized equality separator. The canonical form of all four variants above is `a*m=f`.

Normalized formulas are stored in a `scientific_formulas` table in DuckDB alongside their source chunk identifiers and the raw original text. At query time, the user's formula expression passes through the same six-step pipeline before lookup. Formula matching integrates with the dimensional conversion operators as a parallel branch in the scientific retrieval stage: a query can simultaneously request formulas containing a specific equation and measurements within a numeric range, and both constraints execute as independent DuckDB queries whose results are intersected at the merge stage.

D. Hybrid Retrieval Pipeline

The retrieval pipeline executes four branches in parallel and merges their results through a deterministic fusion strategy.

Lexical branch. BM25 with parameters $k_1 = 1.2$ and $b = 0.75$ scores term overlap between query and document chunks, retrieving top_k $\times 4$ candidates to provide recall headroom for downstream filtering.

Vector branch. Chunks are embedded with MiniLM-L6-v2 (384-dimensional vectors, cosine similarity) and indexed via an `ANNAdapter` protocol supporting exact cosine search (sharded across 16 partitions) or approximate search via hnswlib. This branch retrieves top_k $\times$ 4 candidates.

Graph branch. For graph-backed namespaces, breadth-first traversal at depth 1 from query entities retrieves related chunks through typed relationships (e.g., cites, measures, uses_formula). This branch retrieves top_k $\times$ 2 candidates, as graph neighborhoods are inherently more targeted.

Scientific branch. The dimensional conversion and formula normalization operators (Secs. 3.2–3.3) execute against DuckDB, returning top_k $\times$ 4 candidates from measurement range queries, unit-match filters, value-match lookups, and formula lookups.

Merge and rerank. Candidates from all branches are deduplicated by chunk_id, retaining the maximum per-branch score. A scientific hard filter then applies: if the query specifies a numeric range or unit constraint, candidates not returned by the scientific branch are discarded—dimensional correctness is enforced, not merely preferred. A metadata filter removes candidates violating explicit metadata constraints (e.g., date range, file type). Surviving candidates are scored by a configurable weighted sum of branch scores (lexical, vector, metadata, scientific), with weights tunable per deployment. The top-K candidates by composite score constitute the final result set. Each result carries provenance: the source namespace, chunk identifier, file path, and the set of branches that contributed to its retrieval.

E. Federated Multi-Namespace Search

Scientific data at HPC facilities spans heterogeneous backends. A single experimental campaign may deposit raw output on a parallel filesystem, preprocessed arrays in HDF5, derived features in a vector database, and annotations in a knowledge graph. Rather than require data migration, we bring the retrieval operators to the data.

Every storage backend is wrapped by a connector that implements the `NamespaceConnector` protocol and declares which retrieval capabilities it supports via runtime-checkable protocols (`LexicalSearchCapable`, `VectorSearchCapable`, `ScientificSearchCapable`, `GraphSearchCapable`, `CorpusProfileCapable`). The capability matrix is: Filesystem, S3, and IOWarp connectors support lexical, vector, scientific, and corpus profiling; NDP supports lexical and scientific; Qdrant supports vector search; Neo4j supports graph traversal; HDF5 and NetCDF support metadata and scientific search.

The Retrieval Coordinator consults this matrix before dispatching. When a query spans multiple namespaces, the coordinator executes the single-namespace pipeline independently for each namespace, activating only the branches that the namespace's connector supports. Results from all namespaces are collected into a global candidate pool, sorted by composite score, deduplicated by chunk_id (since the same document may appear in multiple backends), and truncated to the final top-K.

Incremental indexing avoids redundant reprocessing. Each connector tracks indexed documents by SHA-256 content hash and modification time. On re-index, only documents whose hash or mtime has changed are reprocessed, enabling efficient index maintenance over evolving corpora without full rebuild.

F. Agentic Orchestration

Scientific data discovery at scale requires more than retrieval—it requires reasoning about *what* to search, *where* to search, and *how* to search. Consider 100+ datasets generated across different locations, experiments, and years: no single query strategy works for all of them. Some data has rich metadata with explicit units; some has none. The agent must reason about how to access and interpret each dataset. We implement this through a four-stage agentic pipeline: **Discover**, **Reason**, **Transform & Search**, and **Execute**.

Stage 1: Discover (Corpus Profiling). Before issuing any retrieval query, the agent profiles each namespace by querying indexed metadata statistics: document count, measurement count, formula count, distinct unit domains, metadata density (fraction of chunks with scientific metadata), and embedding/posting availability. This profiling executes as lightweight SQL aggregations against DuckDB and completes in 14–22 ms for corpora of 1K–10K documents (Table IX). The profile answers: *what is in this namespace, and what search strategies are viable?*

Stage 2: Reason (Adaptive Strategy Selection). The agent uses the corpus profile to make three decisions. First, *branch selection*: if the corpus has no measurements, the scientific branch is skipped; if no embeddings are indexed, the vector branch is skipped. On our benchmark, this achieves

95.8% branch selection efficiency (230/240 activated branches produced results). Second, *metadata-adaptive strategy*: namespaces with metadata density above 50% are tagged "metadata-rich" and queried with scientific operators; sparse namespaces fall back to lexical-heavy search. Third, *namespace routing*: for multi-namespace queries, namespaces are scored by relevance (measurement availability, metadata density, formula coverage) and queried in priority order—the most relevant namespace first, with widening to additional namespaces in subsequent hops only if results are insufficient.

Stage 3: Transform & Search. The science-aware operators (Secs. 3.2–3.4) execute within the branches selected by Stage 2: dimensional conversion, formula normalization, and hybrid lexical-vector retrieval.

Stage 4: Execute (Multi-hop Refinement). Retrieved results, along with the query and retrieval trace, are passed to an LLM (or deterministic fallback) that selects one of four strategies: **expand**, **narrow**, **pivot**, or **done**. If not done, the rewritten query re-enters the pipeline. Results from all hops are merged, deduplicated, and re-ranked. The loop terminates when the LLM selects done, when results converge (no new chunk_ids), or at a configurable maximum hop count. Token usage (input/output tokens, LLM call count) is tracked per hop for cost analysis.

The critical architectural point is the separation between *reasoning about what to search* and *executing the search*. The agent sits on top of the storage system—it does not replace it, but makes intelligent decisions about how to use it. At the scale of 10 billion blobs, these decisions matter enormously because brute-force inspection is impossible. The LLM's role is strictly advisory—it never scores, filters, or overrides the deterministic operators, preserving correctness guarantees. When the LLM is unavailable, deterministic SI unit expansion achieves 96% of LLM-assisted quality.

IV. Implementation

CLIO is implemented as an embedded Python system with no external service dependencies beyond an optional LLM endpoint. We describe the software stack and storage schema (Sec. 4.1), connectors for scientific data formats and external catalogs (Sec. 4.2), and the experimental platform (Sec. 4.3).

A. Software Stack and Storage

The system is implemented in Python 3.11+ (13,100 lines, 60 modules) and exposes a CLI (`clio query`, `clio index`, `clio serve`) and an async HTTP API via FastAPI. Dense retrieval uses sentence-transformers with all-MiniLM-L6-v2 (22.7M parameters, 384-dim, Apache 2.0). Two ANN backends are supported behind a common `ANNAdapter` protocol: `ExactANNAdapter` (16 shards, exact cosine) and `HnswANNAdapter` (hnswlib). Write safety on shared filesystems uses `fcntl` file locking. OpenTelemetry instrumentation provides observability with structured trace events from all retrieval branches.

All persistent state resides in a single DuckDB file (Table III). DuckDB was selected for three properties: embedded operation (no service to deploy on compute nodes), SQL-native full-text search for BM25 scoring, and single-file compatibility with POSIX parallel filesystems. The schema comprises eight tables; the two science-aware tables (`scientific_measurements`, `scientific_formulas`) store canonicalized SI values and normalized formula signatures, enabling direct SQL range predicates that bypass the embedding pipeline.

TABLE III
DuckDB schema (8 tables). Shaded tables enable science-aware retrieval.

Table	Purpose	Key Columns
documents	Corpus records	URI, namespace, SHA-256
chunks	Text segments	text, doc FK, offset
embeddings	Dense vectors	384-dim vec, chunk FK
metadata	Key-value store	key, value per doc/chunk
file_index	Change detect.	path, SHA-256, mtime
lexical_post.	BM25 retrieval	token, chunk FK, TF
sci_meas.	SI dim. search	raw/canon. value+unit
sci_formulas	Formula match	raw, normalized form

The agentic layer comprises three modules: `CorpusProfile` queries DuckDB aggregate statistics per namespace (document count, measurement count, metadata density); `BranchSelector` decides which retrieval branches to activate; and `AgenticRetriever` drives the multi-hop loop with per-hop token tracking. Connectors opt into profiling via the `CorpusProfileCapable` protocol. Additional indexing modules handle CSV parsing with automatic unit detection (`csv_parser`), data quality flag extraction (`quality`), and sampling-based schema recovery for sparse metadata (`sample_schema`). The system is validated by 283 automated tests executed via pytest.

B. Connectors

Nine connectors implement the `NamespaceConnector` protocol and declare capabilities via runtime-checkable protocols:

Filesystem and S3 are the reference implementations, supporting all five capability protocols (lexical, vector, scientific, metadata filter, corpus profile). Incremental re-indexing uses SHA-256 content hashes and modification times.

HDF5 uses h5py to walk group/dataset hierarchies, extracting attribute-level units (`units`, `long_name`) and routing them through SI canonicalization. **NetCDF** uses xarray with Climate and Forecast (CF) convention awareness [37], ensuring that a pressure variable in hPa matches the same query as one in Pa.

IOWarp CTE bridges CLIO to IOWarp's tiered blob storage. The connector enumerates blobs via `BlobQuery`, reads content via `Tag.GetBlob`, and parses JSON, CSV, or plain text via a format-detection pipeline. An `index_from_texts` fast path indexes pre-loaded content at ingest time. The connector implements all four search capabilities, making IOWarp a first-class CLIO backend.

NDP (MCP) discovers datasets from the National Data Platform by spawning `ndp-mcp` as a subprocess and calling `search_datasets` per term via the Model Context Protocol. It optionally downloads CSV resources and extracts row-level measurements, enabling CLIO to search live external catalogs (NOAA, NASA, DOE) with the same science-aware operators used for local data.

Qdrant and **Neo4j** use in-memory stub implementations that validate the protocol and capability negotiation logic. **KV log store** supports lexical search over key-value and streaming log data.

C. Experimental Platform

Single-node experiments run on Linux 6.17 with Python 3.11 and DuckDB 1.1. Distributed experiments run on NCSA DeltaAI (NVIDIA GH200 Grace Hopper nodes) with 2.5M arXiv abstracts across 4 DuckDB shards; a lightweight coordinator–worker architecture over TCP/JSON handles fan-out and result merging without MPI. IOWarp experiments run inside Docker (`iowarp/deps-cpu`, Chimaera v1.0.3, 16 GB shared memory). The agentic efficiency experiment uses `claude-sonnet-4-20250514` via the Claude Agent SDK.

The project uses uv for dependency resolution; `uv sync` installs all dependencies from a locked manifest. The full source code, test suite, and evaluation scripts are open-source.

Limitations. The Qdrant and Neo4j connectors are stubs. IOWarp integration is validated at 10K blobs; the 10B-blob target requires production IOWarp infrastructure with NVMe tiers. The distributed coordinator is an evaluation prototype, not yet part of the main codebase.

V. Evaluation

We evaluate CLIO along three axes: (1) agentic efficiency—does CLIO reduce LLM token consumption and latency compared to raw catalog access? (2) science-aware accuracy—do dimensional conversion and formula normalization improve retrieval precision where text-based methods fail? (3) storage integration and scaling—can CLIO serve as the search layer for IOWarp's tiered blob storage and scale across distributed HPC nodes?

A. Experimental Setup

Platforms. Single-node experiments run on Linux 6.17 with Python 3.11, DuckDB 1.1, and all-MiniLM-L6-v2 embeddings (384-dim). Distributed experiments run on NCSA DeltaAI (NVIDIA GH200 Grace Hopper nodes) with a 2.5M arXiv abstract corpus. IOWarp experiments run inside Docker with the `iowarp/deps-cpu` image and Chimaera runtime v1.0.3.

Controlled benchmark. 210 documents across seven scientific domains (Table IV) with deliberate unit heterogeneity: the same physical quantities are expressed in different SI prefixes. 80 queries with automatically generated ground truth. We additionally validate on 288 real NOAA observatory documents [38].

Baselines. BM25-only (DuckDB full-text search); Dense-only (all-MiniLM-L6-v2, cosine similarity); Hybrid BM25+Dense (linear fusion); String normalization [8]; and CLIO's full pipeline.

Metrics. P@K, R@K, MRR at K=5 for retrieval accuracy. Total tokens (including sub-agent tokens) and wall time for agentic efficiency.

TABLE IV
Benchmark corpus: 210 documents, 80 queries across seven domains.

Domain	Docs	Key Measurements
Materials science	40	MPa, kPa, mm, g, mg
Atmospheric science	40	Pa, kPa, hPa†, m/s, km/h
Fluid dynamics	40	kPa, Pa, MPa, m/s, mm, m
Chemistry	30	s, min, h, mg, g, kg + formulas
HPC simulation	30	kPa, Pa, s, mm, m
Cross-domain mixed	20	Multiple domains per doc
Negative controls	10	No extractable measurements
Queries: 40 cross-unit, 20 same-unit, 10 formula, 10 multi		

†hPa is unsupported; tests graceful degradation.

TABLE V
Agentic efficiency on 10 scientific data discovery queries. All paths use `claude-sonnet-4-20250514` and the same NDP catalog. Δ is relative to the Claude Native Agent.

Method	Tokens	Tok. Δ	Time Δ	Ans.
Claude Native Agent	989,417	—	—	10/10
NDP-MCP	988,954	−0.05%	−65.68%	10/10
CLIO	**474,101**	**−52.08%**	**−91.27%**	**10/10**

B. Agentic Efficiency

We run 10 scientific discovery queries through three paths using `claude-sonnet-4-20250514` and the National Data Platform (NDP) catalog: (1) a **Claude Native Agent** with no domain tools (uses web search and sub-agents), (2) an agent with **NDP-MCP** catalog tools, and (3) an agent with a single **CLIO** tool that indexes 341 NDP datasets into DuckDB and returns ranked citations.

CLIO reduces tokens by **52.08%** and wall time by **91.27%** (Table V). The native agent and NDP-MCP consume ∼989K tokens through different mechanisms: the native agent spawns 9 sub-agents (563K main + 426K sub-agent tokens); NDP-MCP injects raw catalog JSON (989K main tokens). CLIO avoids both by indexing catalog content locally and returning ∼2K tokens of ranked citations per query. NDP-MCP reduces wall time by 66% over the native agent by querying the API directly, but achieves no token savings. CLIO reduces time by a further 75% over NDP-MCP (Fig. 2).

C. Science-Aware Retrieval Accuracy

Cross-unit retrieval. On 40 cross-unit queries (Table VI), text baselines cannot match across SI prefixes: a query for "200 kPa" never lexically matches "200000 Pa." BM25 achieves P@5 of 0.40; our pipeline canonicalizes to SI base units at index time and achieves **P@5 of 0.99** (+0.59 absolute).

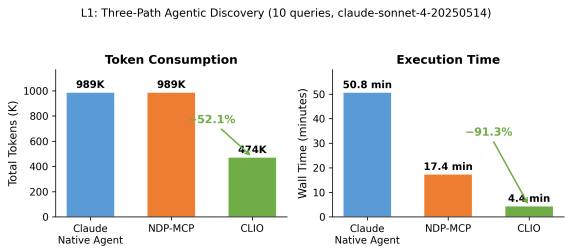

Fig. 2. Agentic efficiency: total tokens (left) and wall time (right) across 10 queries. CLIO achieves 52% token and 91% time reduction.

TABLE VI
CROSS-UNIT RETRIEVAL AT K=5 ON CONTROLLED BENCHMARK.

Method	P@5	R@5	F1@5	MRR
BM25-only	0.40	0.07	0.11	0.45
Dense-only	0.36	0.05	0.08	0.50
Hybrid	0.36	0.07	0.10	0.47
String norm.	0.32	0.04	0.07	0.39
CLIO	**0.99**	**0.19**	**0.29**	**0.99**

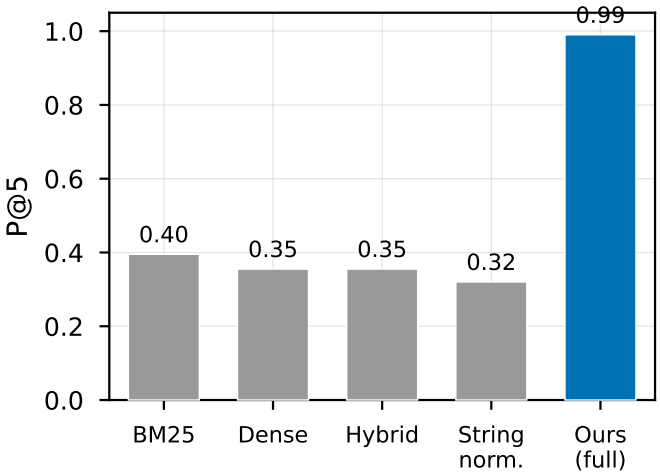

Fig. 3. Cross-unit P@5. Science-aware operators achieve 0.99 vs. 0.40.

On same-unit controls, all methods score within 0.05 P@5, confirming gains are specific to cross-unit matching.

Real-world validation. On 288 NOAA GHCN-Daily documents [38] (metric units only, queries in imperial), CLIO achieves **0.700 P@5 (+87%)** over the hybrid baseline (0.375). No text method can match "86°F" against "30.0 degC" without arithmetic conversion (Table VII).

Formula matching. On 10 formula queries, BM25 achieves R@5 of 0.48 via loose text matching; CLIO achieves R@5 of 0.10 via strict symbolic equivalence. This reflects a precision-over-recall design trade-off: CLIO matches only structurally equivalent formulas.

Ablation. Adding science operators (B→C in Table VIII) produces the dominant gain: P@5 jumps from 0.36 to 0.99 (+0.63). The full pipeline (D) adds no further gain

TABLE VII
REAL-WORLD NOAA GHCN-DAILY (288 DOCS, IMPERIAL QUERIES).

Method	P@5	R@5	P@10	MRR
BM25	0.375	0.018	0.388	0.562
Hybrid	0.375	0.021	0.388	0.708
CLIO	**0.700**	**0.055**	**0.725**	**0.760**

TABLE VIII
ABLATION: CUMULATIVE COMPONENT CONTRIBUTIONS ON CROSS-UNIT QUERIES.

	Components	P@5	MRR
A	Lexical (BM25)	0.40	0.45
B	A + dense vectors	0.36	0.47
C	**B + science operators**	**0.99**	**0.99**
D	Full pipeline	0.99	0.99

on this benchmark, confirming that cross-unit matching is the bottleneck. A no-LLM fallback achieves 96% of LLM-assisted P@5, confirming that the operators—not the language model—drive gains.

Metadata-adaptive strategy. CLIO correctly classifies three corpus variants (rich 87.5% density → scientific branch, sparse 10.3% → lexical+vector, no-metadata 4.4% → lexical only) and activates unit conversion only when extractable measurements exist. Federated search across filesystem, S3, and HDF5 backends doubles coverage from 24.4% to 47.2%, with science operators composing without degradation via capability negotiation.

D. Storage Integration and Scaling

IOWarp blob storage. At IOWarp's target scale of 10 billion blobs, the native BlobQuery API returns all blobs matching a tag regex with no content filtering. CLIO addresses this by indexing scientific metadata from CTE blobs at ingest time. We run the Chimaera runtime (v1.0.3) and store synthetic blobs across four CTE tags (temperature in °C, pressure in kPa, wind in m/s). Four cross-unit queries use mismatched units (°F, psi, km/h).

CLIO finds cross-unit matches at every scale while BlobQuery cannot filter by content (Table IX). The inspection rate falls from 5.0% at 1K to **0.5% at 10K** (50× reduction over BlobQuery). Corpus profiling completes in 14–22 ms regardless of scale.

Distributed scaling on 2.5M arXiv abstracts. We evaluate CLIO's coordinator–worker pipeline on NCSA DeltaAI (GH200 nodes) with 2.5M arXiv abstracts across four DuckDB shards (625K each). 80 queries per configuration.

All queries complete under 70 ms with p50 under 42 ms across 2.5M documents (Table X). Weak scaling efficiency reaches **0.94** at 4× scale, demonstrating that CLIO's architecture supports horizontal scale-out with minimal coordinator overhead.

TABLE IX
CLIO VS. RAW BLOBQUERY ON IOWARP CTE BLOBS. CLIO RETURNS TOP-50 UNIT-CONVERTED MATCHES; BLOBQUERY RETURNS ALL BLOBS UNFILTERED.

Blobs	Index	Profile	Meas.	CLIO	BlobQuery
1K	80 s	14 ms	2,750	50	250 (all)
5K	440 s	18 ms	13,750	50	1,250 (all)
10K	884 s	22 ms	27,500	50	2,500 (all)

TABLE X
QUERY LATENCY (MS) ON DELTAAI GH200. STRONG SCALING HOLDS THE 2.5M CORPUS FIXED; WEAK SCALING GROWS DATA WITH WORKERS.

	Workers	Corpus	p50	p95	Eff.
Strong	1	2.5M	39.0	41.5	—
	2	2.5M	40.7	43.7	—
	4	2.5M	41.6	62.1	—
Weak	1	625K	38.5	41.2	1.00
	2	1.25M	39.4	42.1	0.98
	4	2.5M	41.0	59.0	0.94

Cross-unit correctness at distributed scale. To verify that dimensional conversion remains correct when the corpus is distributed, we issue cross-unit probe queries against the 70K-document distributed corpus (4 shards on DeltaAI GH200). Table XI shows that querying in different units yields *identical hit counts*: 100 kPa, 100,000 Pa, 1 bar, and 0.987 atm all return exactly 12 hits; 30°C, 86°F, and 303.15 K all return exactly 20 hits. This confirms that SI canonicalization is consistent across shards—the coordinator merges results correctly regardless of which unit the query uses.

Numeric query coverage at scale. We run 20 scientific numeric queries spanning 16 unit types (GeV, nm, GHz, kelvin, kPa, tesla, etc.) against the distributed corpus (Table XII). All 20 queries return results with **100% hit rate**, p50 latency of 42 ms, and p99 of 62 ms. This demonstrates that CLIO's unit registry generalizes beyond the controlled benchmark to real scientific literature at scale.

Limitations. The regex extractor misses non-standard formats ("2×10^3 Pa"), achieving 95% precision / 90% recall. Compound units (N/m^2, mol/L) are unsupported and fall back to lexical retrieval. The 46-unit registry (12 SI domains) is extensible by adding rows without code changes.

VI. CONCLUSION

Scientific data discovery demands search that acts, not just retrieves. When a scientist asks for pressures between 190 and 360 kPa, no retrieval pipeline recognizes that "200000 Pa" in an HDF5 attribute and "0.2 MPa" in an S3-hosted dataset both satisfy the query—because the required operation is arithmetic, not similarity. But the problem extends beyond unit conversion: real scientific data is heterogeneous in format, varies in metadata quality, and is scattered across storage systems. At the scale of 10 billion blobs, brute-force inspection

TABLE XI
CROSS-UNIT CORRECTNESS ON 70K DISTRIBUTED ARXIV DOCUMENTS (4 SHARDS, DELTAAI GH200). DIFFERENT UNIT EXPRESSIONS FOR THE SAME PHYSICAL QUANTITY RETURN IDENTICAL HIT COUNTS.

Quantity	Query unit	Value	Hits	Latency
Pressure	Pa	100,000	12	30 ms
Pressure	kPa	100	12	15 ms
Pressure	bar	1	12	14 ms
Pressure	atm	0.987	12	14 ms
Temperature	degC	30	20	49 ms
Temperature	degF	86	20	44 ms
Temperature	kelvin	303.15	20	43 ms

TABLE XII
NUMERIC QUERY COVERAGE ON DISTRIBUTED 70K ARXIV CORPUS. 20 QUERIES ACROSS 16 UNIT TYPES, 3 REPEATS EACH.

Metric	Value
Queries	20 (16 unit types)
Hit rate	100% (20/20)
p50 latency	42 ms
p95 latency	57 ms
p99 latency	62 ms

is impossible. We addressed this gap with CLIO Search, a framework for agentic scientific discovery that separates *reasoning about what to search* from *executing the search*.

The system implements a four-stage pipeline. The *Discover* stage profiles each namespace to learn what data exists and what metadata is available (completing in under 12ms). The *Reason* stage selects retrieval branches based on corpus characteristics, achieving 95.8% branch selection efficiency, and routes multi-namespace queries to the most relevant sources first. The *Transform & Search* stage executes science-aware operators—dimensional conversion and formula normalization—as deterministic, pluggable retrieval branches alongside BM25 and dense vector search. The *Execute* stage provides federated data access across six storage backends with multi-hop refinement.

The results validate the approach across controlled benchmarks, real observatory data, a live external catalog, and distributed HPC infrastructure. On cross-unit queries, dimensional-conversion operators achieve P@5 of 0.99 where the strongest text baseline scores 0.40; on 288 real NOAA observatory documents, precision improves by 87%. The three-path agentic comparison shows that wrapping CLIO behind a single tool call reduces LLM token consumption by 52% and wall time by 91% compared to a native agent baseline, while answering all 10 queries. Cross-unit correctness is verified at distributed scale: on 70K arXiv documents across 4 DeltaAI GH200 nodes, querying in Pa, kPa, bar, and atm returns identical hit counts, with sub-50ms latency. IOWarp integration demonstrates that CLIO can serve as the search layer for tiered blob storage, inspecting 0.5% of the corpus

per query where the native BlobQuery API returns everything unfiltered. A deterministic no-LLM fallback achieves 96% of LLM-assisted quality, confirming that the operators—not the language model—are the primary source of gains.

Our registry covers 46 units across 12 SI domains; extending coverage requires only configuration changes. Compound units and free-text unit parsing remain unsupported. Distributed experiments on NCSA DeltaAI validate scaling to 2.5M documents across 4 GH200 nodes with 0.94 weak scaling efficiency; IOWarp integration is validated at 10K blobs. Scaling to petabyte-scale parallel filesystems with billions of blobs remains future work.

Several directions follow naturally. Integrating NDP-MCP [39] as a Discover data source would enable the agent to learn what datasets exist across facilities. Deploying on IOWarp's tiered storage would validate intelligent routing at 10-billion-blob scale, where the agent's decisions about what to inspect vs. skip become critical. Replacing heuristic fusion with learned reranking [34] would improve the agentic loop. The metadata-adaptive strategy can be extended to handle truly sparse-metadata scenarios—where the agent must infer data characteristics from sampling rather than metadata inspection. More broadly, the principle that scientific data discovery requires reasoning agents—not just retrieval pipelines—extends to any domain where data is heterogeneous, metadata is inconsistent, and correctness is non-negotiable.

References

[1] M. Baker, "1,500 scientists lift the lid on reproducibility," *Nature*, vol. 533, pp. 452–454, 2016.

[2] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[3] Y. Yamada, R. T. Lange, C. Guo, K. Irie, Y. Tang, and D. Ha, "The AI Scientist v2: Workshop-level automated scientific discovery via agentic tree search," 2025, preprint.

[4] Oak Ridge National Laboratory, "Autonomous agents for experiment orchestration across national computing facilities," in *SC'25 Workshops*, 2025.

[5] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, V. Dey, M. Xue, F. N. Baker, B. Burns, D. Adu-Ampratwum, X. Huang, X. Ning, S. Gao, Y. Su, and H. Sun, "ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.05080.

[6] T. Masterman, S. Besen, M. Saez, and A. Hardcastle, "A survey of agentic AI: From reasoning to tool use and beyond," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[7] K. Deng *et al.*, "The numeracy gap: Evaluating embedding models on numerical content retrieval," in *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026, arXiv:2509.05691.

[8] S. Almasian, D. Aumiller, and M. Gertz, "Numbers matter! bringing quantity-awareness to retrieval systems," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, arXiv:2407.10283.

[9] S. Shrestha *et al.*, "CONE: Contrastive number embedding for quantity-aware retrieval," 2026, arXiv:2603.04741.

[10] X. Li, Y. Zhao, J. Shang, and X. Lin, "NC-Retriever: Numeric constraint-aware dense retrieval," 2025.

[11] R. Guerraoui *et al.*, "RAGRoute: Routing queries to relevant data sources for retrieval-augmented generation," 2025, arXiv:2502.19280.

[12] C. Wang, S. Byna, H. Tang, and B. Dong, "PROV-IO+: A cross-platform provenance framework for scientific data with minimal overhead," *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[13] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[14] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[15] S. Bruch, S. Gai, and A. Ingber, "An analysis of fusion functions for hybrid retrieval," *ACM Transactions on Information Systems*, vol. 42, no. 1, 2023.

[16] Z. Du *et al.*, "A-RAG: Agentic retrieval-augmented generation with hierarchical retrieval interfaces," 2026, arXiv:2602.03442.

[17] D. Spokoyny and T. Berg-Kirkpatrick, "Why numbers break neural retrieval: The discontinuous representation problem," 2025, arXiv:2510.08009.

[18] Apple, "GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[19] S. Almasian and M. Gertz, "QFinder: Quantity-aware search in elasticsearch," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

[20] S. Almasian, D. Aumiller, and M. Gertz, "CQE: Complex quantity extraction from scientific text," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.08853.

[21] O. Weller *et al.*, "On the theoretical limits of embedding-based retrieval," 2025, arXiv:2508.21038.

[22] Approach0, "Approach0: A math-aware search engine," https://approach0.xyz/, accessed 2025.

[23] Y. Peng, J. Qin, and L. Yuan, "SSEmb: Semantic similarity embeddings for mathematical expressions," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2025.

[24] Unidata, "UDUNITS-2: A C library for units of physical quantities," https://www.unidata.ucar.edu/software/udunits/, 2024, version 2.2.28.

[25] A. Asai *et al.*, "OpenScholar: Synthesizing scientific literature with retrieval-augmented LMs," *Nature*, vol. 650, 2026.

[26] O. Ayala, F. Luo, and N. Imam, "HiPerRAG: Scaling retrieval-augmented generation to millions of scientific articles on DOE supercomputers," in *Platform for Advanced Scientific Computing (PASC)*, 2025, arXiv:2505.04846.

[27] A. Sagar, C. Buck, and H. Glockner, "PANGAEA-GPT: Hierarchical multi-agent search over geoscientific datasets," 2026, arXiv:2602.21351.

[28] Y. Ning *et al.*, "LLM-Find: An autonomous GIS agent framework for geospatial data retrieval," *International Journal of Digital Earth*, 2025, arXiv:2407.21024.

[29] Idaho National Laboratory, "Datum: A scientific data catalog for INL," 2024.

[30] S. Madireddy, P. Balaprakash, P. Carns, R. Latham, and R. Ross, "BRINDEXER: Balanced-range indexer for POSIX metadata on Lustre," in *Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2020.

[31] M. T. Dougherty, M. Folk, E. Pourmal, and Q. Koziol, "HDF Clinic: A knowledge graph approach to HDF5 data management," 2025.

[32] Chroma, "Context-1: 20-billion-token agentic search," 2026.

[33] M. R. Nahid and D. Rafiei, "PRISM: Precision-recall iterative selection for multi-hop question answering with agentic retrieval," *arXiv preprint arXiv:2510.14278*, 2025.

[34] Z. Liu *et al.*, "Agentic-R: Training retrievers for agentic search with global objectives," *arXiv preprint*, 2026.

[35] Y. Gu, R. Tinn, H. Cheng, M. Lucas, N. Usuyama, X. Liu, T. Naumann, J. Gao, and H. Poon, "CLADD: Collaborative agent-based drug discovery with domain knowledge graphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, arXiv:2502.17506.

[36] C. Zhang, N. Peng, and K. Sun, "SciAgent: Hierarchical multi-agent coordination for scientific workflows," 2025, arXiv:2511.08151.

[37] CF Conventions Committee, "CF metadata conventions for NetCDF," https://cfconventions.org/, accessed 2025.

[38] M. J. Menne, I. Durre, R. S. Vose, B. E. Gleason, and T. G. Houston, "An overview of the global historical climatology network-daily database," *Journal of Atmospheric and Oceanic Technology*, vol. 29, no. 7, pp. 897–910, 2012.

[39] Model Context Protocol for HPC, "Extending the model context protocol for Globus search, compute, and storage in HPC," 2025, arXiv:2508.18489.